

SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection. Answer:- Testing as an Engineering Activity Testing should be treated as a systematic engineering activity rather than random execution of test cases. The team should first analyze requirements, identify risks, design test cases, define expected results, execute tests, and document defects. Structured techniques such as equivalence partitioning and boundary value analysis provide better coverage. Test cases should be reviewed and prioritized based on risk and business importance. This approach makes testing repeatable and measurable. As a result, defects are detected earlier and overall software quality improves.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples. Answer:- Well-Defined Testing Process in Banking	3

	A well-defined testing process provides a consistent sequence of planning, designing, executing, reporting, and closing testing activities. In a banking system, requirements such as account validation, fund transfer, withdrawal limits, and transaction security must be tested systematically. Different testing levels such as unit, integration, system, and regression testing help identify defects at appropriate stages. Proper documentation also makes testing traceable and repeatable. For example, transaction failures can be detected before deployment through integration and system testing. Thus, a defined process improves reliability, security, and customer confidence.	
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes. Answer:- Testing as a Structured SDLC Process Testing should be performed throughout the SDLC rather than only after development is completed. During requirements analysis, testers identify ambiguities and create acceptance criteria. During design and coding, unit and integration testing verify individual components and their interactions. System testing validates the complete e-commerce application, while regression testing ensures that new changes do not break existing features. Each phase should have defined entry and exit criteria. Process discipline therefore reduces integration defects and produces more predictable quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples. Answer:- Error, Defect and Failure	3

	An error is a human mistake made by a developer, analyst, or tester. When an error is introduced into the software and causes the software to behave incorrectly, it becomes a defect or bug. When the defect is executed and the system produces an incorrect or unexpected result, it results in a failure. For example, a developer may incorrectly write a login condition, creating a defect. When a valid user is rejected during execution, the observed incorrect behavior is a failure. Thus, error can introduce a defect, and execution of a defect can cause failure.	
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example. Answer:- Early Testing Principle:- The principle of early testing states that testing should begin as early as possible in the software lifecycle. Requirements, design documents, and prototypes can be reviewed before coding begins. This helps identify errors before they become expensive defects. For example, an incorrect payment requirement found during requirement review is much cheaper to fix than a payment defect discovered after deployment. Early testing also reduces rework and development effort. Therefore, involving testers early improves defect detection efficiency and overall project quality.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement. Answer:- Tester Involvement During Requirements Testers should participate in requirement analysis to identify ambiguity, inconsistency, incompleteness, and missing conditions. They can review requirements for clarity, testability, and measurable acceptance criteria. In	3

	a healthcare system, a tester may identify an unclear requirement such as “patient data should be stored securely” and request specific security criteria. Early reviews prevent misunderstandings from reaching design and implementation. Testers can also prepare initial test scenarios from the requirements. Therefore, early testing involvement reduces defects and development cost.	
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues. Answer:- Requirement Defects in Software Lifecycle Unclear requirements can introduce defects at the earliest stage of the software lifecycle. If developers interpret an ambiguous payroll rule differently from the business requirement, the implemented calculation may produce incorrect salary values. These errors can then propagate into design, coding, and testing. Requirement reviews, stakeholder discussions, prototypes, and clear acceptance criteria can identify such problems early. Requirements should be complete, consistent, unambiguous, and testable. Clear requirements therefore prevent defects from spreading into later development stages.	3
8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue. Answer:- Boundary Value Analysis A system crash caused by boundary inputs indicates a boundary-related functional defect, often resulting from inadequate boundary testing. Testers may have tested only normal values and ignored minimum, maximum, just-below, and just-above boundary values. For example, if an	3

	age field accepts 18–60, values such as 17, 18, 19, 59, 60, and 61 should be tested. Boundary Value Analysis specifically targets errors occurring at the edges of valid ranges. Proper boundary testing could therefore identify the crash before release.	
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality. Answer:- Defect Repository and Test Case Design A defect repository stores information about previously identified defects, including their causes, affected modules, severity, and resolution. Testers can analyze historical defects to identify recurring problem areas and create targeted test cases. For example, if previous releases repeatedly had payment calculation defects, additional payment and transaction test cases should be included in regression testing. The repository also helps identify patterns and high-risk modules. Reusing this knowledge prevents repeated mistakes. Thus, a defect repository improves test coverage and software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented. Answer: Functional Defect in ATM An ATM failure during withdrawal caused by incorrect transaction logic is a functional defect because the system does not perform a required business function correctly. For example, the ATM may deduct money from an account without dispensing cash or may allow withdrawal beyond the available balance. Functional test cases should cover valid withdrawals, insufficient balance, invalid amounts, daily limits, and	3

	transaction failures. Integration testing should also verify communication between the ATM, banking server, and account database. Proper functional and integration testing could have detected the defect before production.	
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency. Answer:- Defect Clustering Defect clustering means that a small number of modules often contain a large proportion of the defects. The testing team should analyze historical defect data to identify modules with high defect density. These modules should receive greater testing effort, deeper reviews, and more regression testing. For example, if most defects occur in the payment and authentication modules, they should receive higher priority than stable modules. This allows limited testing resources to be used where they provide the greatest benefit. Therefore, defect clustering improves testing efficiency and risk management.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution. Answer:- Performance Defect Slow performance under heavy user load is classified as a performance or non-functional defect. The application may function correctly for a small number of users but fail to meet response-time or throughput requirements under high load. Load testing, stress testing, and endurance testing can identify such problems. For example, an e-commerce website should be tested with thousands of simultaneous users before a major sale. Classification helps testers select the correct testing technique and	3

	investigation method. This leads to faster diagnosis and appropriate performance improvements.	
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency. Answer:- Historical Defect Data for Regression Testing Historical defect data can be reused to design effective regression test suites. Testers should identify previously defective modules, frequently changed functionality, and defects that have repeatedly reappeared. Corresponding test cases should be included in every regression cycle. For example, if previous releases had payment and login failures, these areas should receive priority after every major change. Historical data also helps prioritize high-risk tests when execution time is limited. Thus, defect history improves coverage while reducing unnecessary testing effort.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps. Answer:- Tester Role in Integration Defects Testers help reduce integration defects by becoming involved before integration testing begins. They can understand component interfaces, data formats, dependencies, and communication protocols during the development phase. Testers should discuss integration risks with developers and prepare integration scenarios early. For example, they can verify whether the order module sends the correct customer and payment information to the payment module. Early collaboration exposes interface misunderstandings before they become expensive defects. Therefore,	3

	communication between developers and testers improves integration quality.	
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality. Answer:- Defect Prevention Defect prevention focuses on removing the causes of defects rather than simply finding and fixing them after implementation. The team should perform root-cause analysis on recurring defects and identify process weaknesses. Code reviews, requirement reviews, coding standards, developer training, automated testing, and improved processes can prevent repeated problems. For example, repeated calculation defects may indicate unclear requirements or inadequate unit testing. Correcting the underlying cause prevents similar defects in future releases. Therefore, defect prevention improves long-term software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability. Answer:- Structured Testing for Mobile Applications A mobile application requires a standardized testing strategy covering functionality, compatibility, usability, performance, security, and different devices. The team should create test plans, reusable test cases, defined test data, and clear defect-reporting procedures. Testing should be performed across different operating systems, screen sizes, network conditions, and device configurations. Regression testing should be executed after significant changes. Random testing alone cannot provide reliable	3

	coverage. A structured approach makes testing consistent and reduces application crashes and compatibility problems.	
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality. Answer:- Testing vs Debugging Testing and debugging have different purposes. Testing is the process of executing or evaluating software to identify defects, while debugging is the process of finding the cause of a detected defect and correcting the code. For example, testing an ATM may reveal that money is deducted without cash being dispensed. The tester reports the failure with appropriate evidence. The developer then debugs the transaction logic, identifies the incorrect code, and fixes it. Testing verifies that the defect exists, while debugging determines and removes its cause.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled. Answer:- Cross-Browser Testing Cross-browser testing verifies that a web application works consistently across different browsers and versions. The team should identify supported browsers such as Chrome, Firefox, Edge, and Safari and create a compatibility test matrix. Important functions such as login, payment, navigation, forms, and page layout should be tested on each supported environment. Responsive behavior and JavaScript/CSS compatibility should also be checked. Automated browser testing can reduce repetitive effort. This approach helps identify browser-specific defects before customers encounter them.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability. Answer:- Validation in Logistics Application Improper validation can allow incorrect, incomplete, or unexpected data to enter the system and produce inconsistent results. Testers should identify input validation rules and create positive, negative, invalid, and boundary test cases. For example, a logistics system should reject invalid shipment IDs, negative quantities, incorrect addresses, and unsupported delivery dates. Requirements should clearly define acceptable input and expected output. Early validation testing prevents invalid data from propagating between modules. This improves reliability and consistency of the application.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design. Answer:- Systematic Test Design Random testing without a defined strategy provides unpredictable and incomplete coverage. Testing as an engineering activity requires requirements analysis, test planning, systematic test case design, execution, defect reporting, and evaluation. Techniques such as equivalence partitioning, boundary value analysis, decision tables, and state-transition testing help select meaningful test cases. Test cases should have clear inputs, expected outputs, and pass/fail criteria. Risk-based prioritization can focus effort on critical functionality. Systematic design therefore detects more defects with better use of testing resources.	3

21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency. Answer:- Data Consistency in Integration Data inconsistency between healthcare modules usually indicates an integration or data-handling defect. Testers should verify data flow between modules, interfaces, APIs, databases, and external systems. The same patient information should maintain correct values, formats, and identifiers throughout the workflow. Integration test cases should cover valid data, invalid data, concurrent updates, transaction failures, and synchronization issues. Logs and database comparisons can help identify where data changes incorrectly. Proper integration testing ensures accurate and consistent information across the system.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality. Answer:- Testers Across the SDLC Testers should contribute throughout the SDLC rather than entering only after coding is complete. During requirements, they review ambiguity and testability; during design, they identify risks and integration concerns; during development, they support unit and integration testing. During system testing, they validate complete functionality, and during maintenance they perform regression testing. Early tester involvement improves communication between developers, business stakeholders, and testers. It also identifies defects before implementation becomes expensive to change. Therefore, testing becomes a continuous quality activity.	3

23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures. Answer:- Test Coverage and Peak Usage Failures during peak usage often occur because important scenarios were missing from the test suite. Testers should analyze requirements and risks to identify normal, exceptional, concurrent, and high-load scenarios. Load, stress, performance, and scalability testing should be performed where appropriate. For example, an online ticket system should be tested with a large number of users accessing the same event simultaneously. Code coverage alone is not sufficient; functional and risk coverage are also important. Better coverage reduces the probability of critical production failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis. Answer:- Defect Repository and Documentation A defect repository should contain complete information such as defect ID, description, severity, priority, environment, steps to reproduce, expected result, actual result, status, and resolution. This information allows developers and testers to track defects throughout their lifecycle. Historical records can be analyzed to identify recurring defects and high-risk modules. Proper documentation also prevents duplicate defect reports and improves communication. Management can use repository data to measure defect trends. Therefore, systematic defect documentation improves tracking, analysis, and quality management.	3

25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability. Answer:- Structured Testing for Banking Systems Banking applications require highly reliable and systematic testing because transaction failures can directly affect customers and financial records. Testing should begin with requirement reviews and continue through unit, integration, system, security, performance, and regression testing. Critical functions such as deposits, withdrawals, transfers, authentication, and transaction rollback require extensive coverage. Risk-based testing should prioritize high-impact financial operations. Automated regression tests can verify existing functionality after changes. Structured testing therefore reduces repeated failures and improves banking-system reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection. Answer:- Edge Case and Boundary Testing Testing only normal inputs leaves the system vulnerable to failures at the limits and unusual conditions. Testers should include minimum, maximum, just-inside, just-outside, empty, null, and unexpected values where applicable. For example, if a field accepts values from 1 to 100, values such as 0, 1, 2, 99, 100, and 101 should be tested. Boundary Value Analysis is effective because defects frequently occur at input boundaries. Including these cases improves test coverage and increases the probability of detecting hidden defects.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing	3

	approaches to address this issue. Explain how standardization improves outcomes. Answer:- Process-Based Testing Process-based testing establishes a common testing methodology across all modules and teams. The organization should define standard procedures for test planning, test-case design, execution, defect reporting, regression testing, and closure. Common templates and quality criteria ensure that different testers follow the same expectations. Metrics such as test coverage, defect density, and defect leakage can be used to measure results. This reduces inconsistency between modules. Standardization therefore produces more predictable and uniform testing outcomes.	
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness. Answer:- Input Validation and Robustness Unexpected input handling should be tested systematically using valid, invalid, boundary, null, empty, and unexpected data. Testers should verify that the application rejects invalid inputs safely and provides meaningful error messages instead of crashing. For example, a numeric field should be tested with letters, negative numbers, extremely large values, and empty input. Both client-side and server-side validation should be verified where applicable. Negative testing is particularly important for robustness. Proper validation prevents crashes and improves system reliability.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage. Answer:-	3

	Reducing Defect Leakage Defect leakage occurs when defects escape testing and reach production. The root causes may include incomplete requirements, inadequate test coverage, poor test design, insufficient regression testing, or weak communication. Teams should analyze escaped defects using root-cause analysis and update test cases accordingly. Risk-based testing, automation, code reviews, requirement reviews, and regression testing can reduce leakage. Production defects should become learning inputs for future testing. This creates a continuous improvement cycle and improves release quality.	
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency. Answer:- Early Defect Detection The testing strategy should be redesigned around the principle of early testing. Testers should participate in requirements and design reviews, developers should perform unit testing, and integration testing should occur before complete system testing. Static reviews and automated tests can detect problems without waiting for the final stage. Defects found early generally require less rework because fewer dependent components have been built around them. For example, correcting an ambiguous requirement during analysis is cheaper than correcting the resulting code and database changes later. Therefore, early detection reduces cost, effort, and schedule delays.	3

Code of Conduct:

I (P.Veeraswami / 192311253) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Veeraswami



SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand